

3.6 리스트 (List)

블록코딩에서 리스트라고 불렀던 저장공간의 목적은 무엇이었는가? 앞에서 변수가 단 하나의 변화하는 값(또는 사전에 값을 특정할 수 없는 값)을 저장했다면, 리스트라는 공간은 변화하는 단 하나의 값이 아니라, 여러 개의 복수의 값을 저장할 수 있는 있는 공간인 것이다.

이러한 공간을 파이썬에서는 리스트(list)라고 부르고, 다른 텍스트코딩 언어에서는 일반적으로 배열(array)라고 부르는데 엔트리에서 이러한 복수 개의 값을 저장하는 공간을 이름지으면서 어떤 다른 단어로 예를들어 '목록' 등의 번역된 한글이름으로 표현할 수 있었음에도 의도적으로 리스트(list)라고 파이썬 상의 용어표현 그대로 도입한 것부터 사실상 이 개념을 이해시키기 위한 잘짜여진 각본이 있었던 생각이 든다.

블록코딩에서 리스트가 엔트리-파이썬 텍스트코딩에서는 어떻게 사용할 수 있을까? 그럼, 예제를 통해 엔트리-파이썬에서 어떻게 코딩할 수 있는지 상세히 알아보도록 하자. 3.3장에서 만들었던 덧셈전용 계산기 프로그램에서 이 리스트를 활용하여 사용성을 더 개선시킬 수 있다. 3.3장의 덧셈전용 계산기는 사용자로부터 입력된 임의의 두 값만을 받아 합계를 구해주고 있다. 이제는 사용자가 입력할 수 있는게 단지 두개의 값만이 아닌 제한 없이 더 많은 수를 연속적으로 입력받아 입력한 모든 수를 더한 합계를 구해줄 수 있다면 사용자 입장에서는 더 나은 사용성을 제공하는 프로그램이 될 것이다.

먼저, 리스트를 어떻게 사용할 수 있는 문법을 확인해보자.

리스트명 = [문자, 숫자 등의 여러 값. 단, 값들은 콤마(,)로 구분]

리스트의 이름은 변수의 이름을 지을 때와 동일하게 임의로 지을 수 있는데, 나를 포함해 누구라도 코드의 의미를 빠르고 효과적으로 이해할 수 있게 하기 위해 항상 의미있게 이름 짓는 습관을 드리는게 필요하다. 이전에도 언급했지만, 이는 프로그래밍 세계에서의 주요한 기본 덕목 중에 하나라고 할 수 있다.

[!NOTE]

리스트명은 임의로 사용할 수 있다고는 했으나, 사실은 파이썬에서 리스트 이름을 지을 때, 변수이름 지을 때와 마찬가지로 크게 지켜야 할 2가지 규칙은 있다.

- 리스트의 이름은 숫자로 시작할 수 없다.
- 리스트의 이름 사이에 공백이 있을 수 없다.

다음은 문법에 맞게 작성한 리스트의 예제들이다.

```
words = ["apple", "banana", "grape"]
```

```
numbers = [1, 3, 10, 25, 2]
```

```
complex = ["apple", 2, 32, "berry", 56, 5]
```

words라는 이름에 리스트는 문자형(string) 데이터 유형(type)의 값(또는 아이템이라 호칭) 3개를 갖고 있다.

numbers라는 이름에 리스트는 숫자형 데이터 유형의 아이템 5개를 갖고 있다.

complex라는 이름에 리스트는 문자형, 숫자형의 데이터 유형의 값을 섞어서 총 6개의 아이템을 갖고 있다.

지금까지 우리는 리스트를 선언하면서 동시에 초기값(선언하면서 동시에 맨 처음 할당하는 값)을 설정하는 법을 배웠다. 이제 더 배워야 할 지식은 리스트 안에 저장된 값들(아이템들)을 접근해 읽어오는 법, 기존 리스트에 새로운 신규 값(아이템)을 추가하거나, 삭제하는 방법에 대해 알 필요가 있다. 방금 언급된 리스트를 다루는 방법들은 이번 장의 서두에서 언급된 우리가 목표하는 앱인 발전된 덧셈계산기 앱의 코드들을 하나씩 분석해 보면서 배워나가기로 하자.

실행결과

더할 숫자를 입력하세요. 현재까지 입력한 값의 합을 알고 싶으면 '='를 입력하세요. 앱을 종료하고 싶으시면 'x'를 입력하세요.



numbers	
0	2
1	3

result 0

=

✓

5



numbers	
---------	--

result 5

블록코딩

A Scratch-style block coding interface for a calculator. The code starts with a green flag click event that triggers a speech bubble saying "당신이 입력한 모든 수를 더해주는 계산기 입니다" (This is a calculator that adds all the numbers you input). This is followed by a 2-second wait block. Then, a "forever" loop begins. Inside the loop, it asks "더할 숫자를 입력하세요. 현재까지 입력한 값의 합을 알고 싶으면 '='를 입력하세요. 앱을 종료하고 싶으시면 'x'를 입력하세요." (Enter a number to add. If you want to know the sum of the numbers entered so far, enter '='. If you want to end the app, enter 'x'). It then checks if the answer is 'x'. If yes, it says "all" and ends the loop. If the answer is '=', it enters a "while" loop that continues as long as the 'numbers' list is not empty. Inside this while loop, it adds the first number in the 'numbers' list to the 'result' and removes it from the list. After the while loop, it prints the result and waits 2 seconds. If the answer is neither 'x' nor '=', it adds the input number to the 'numbers' list. The loop ends with a "say" block.

엔트리-파이썬

```
1 | # 엔트리봇 오브젝트의 파이썬 코드
2 | import Entry
3 |
4 | result = 0
5 | numbers = []
6 |
7 | def when_start():
8 |     Entry.print("당신이 입력한 모든 수를 더해주는 계산기 입니다")
9 |     Entry.wait_for_sec(2)
10 |
11 |     while True:
12 |         Entry.input("더할 숫자를 입력하세요. 현재까지 입력한 값의 합을 알고 싶으면 '='를 입력하세요. 앱을 종료하고 싶으시면 'x'")
13 |
14 |         if Entry.answer() == "x":
15 |             Entry.stop_code("all")
16 |
17 |         if Entry.answer() == "=":
18 |             while len(numbers) != 0:
19 |                 result = result + numbers[0]
20 |                 numbers.pop(0)
21 |
22 |             Entry.print(result)
23 |             Entry.wait_for_sec(2)
24 |             result = 0
25 |         else:
```

34 먼저 5번라인에서 numbers라는 이름에 리스트 한개를 선언한 내용이 좀 특이한데, 초기값이 전혀 없는 상태로 리스트를 생성했다는 것을 알 수 있는데, 왜냐하면 대괄호 안에 안에 아무값도 없는 상태로(그냥 대괄호만 열고 곧바로 닫는 형태로) 선언했기 때문이다. 이렇게 선언한 이유는 남득이 가능한데, 우리 앱은 사용자가 임의로 제한없이 입력하는 숫자값들을 담아놓을 임시공간이 필요하고, 그러한 목적에 부합하는 공간인 리스트를 활용하되 처음에는 사용자가 아무것도 입력하지 않은 상태이고, 이 상태에서도 리스트의 존재는 필요함으로 이런식으로 초기화 했다고 예상할 수 있다.

34 다음으로 26번 라인을 먼저 살펴보자. 26번 라인은 리스트에 사용자가 입력한 숫자값을 신규 아이템으로 추가하는 코드이다. `append`라는 함수를 통해 사용자 입력값을 `Entry.answer()` 통해 읽어 전달하고 있는데 그 `append` 함수의 사용을 위해 **number.**으로 즉, 리스트의 이름 안에 위치한다는 의미로 함수를 호출하고 있는 점이 특이할 것이다. 어떻게 이런식의 함수호출이 가능할까? 이유는 리스트 그 자체가 객체(Object)이기 때문에 그런 것인데, 이 부분을 자세히 설명하기에는 이 책의 범위를 초과하는 것이라 따로 관심있는 사람은 따로 파이썬에서의 객체 다루기에 대해 공부하길 바란다. 현재로서는 리스트에 신규 아이템 추가 방법은 리스트이름.**append**(추가하려는 값) 문법을 통해 사용한다고 알아두면 되겠다.

34 이제 18~20번 라인을 살펴보자. `while`을 사용한 반복문이다. 그런데 언제까지 반복할지에 대한 조건이 있는데, `len(numbers) != 0` 라는 조건이고, 여기서 우리는 기존에 "`==`" 기호(연산자)가 의미하는 것이 무엇인지는 알고있다. 하지만, "`!=`"의 의미는 무엇일까? 바로 "`==`"의 반대 의미로, 좌우의 두 값이 서로 같지 않을 때를 나타내는 의미가 된다. 그렇다면 이 코드의 의미를 해석하면 `len(numbers)` 함수의 리턴값이 0 과 같지 않을 때까지 반복하는 것이다.

이제 `len` 함수가 무엇인지 알아야겠다. 사용법을 보면 특이한데 함수명 앞에 그 함수가 위치한 어떤 위치정보 예를들어 어떤 라이브러리의 이름이 나, 리스트명나 변수명으로 참조하여 함수를 부른 것이 전혀없다. 이 말인 즉은 이 함수는 어디에 속한게 아닌 파이썬 언어 내부에 디폴트로 내장되어 있는 함수라는 것이다. 그래서, 이름만으로 맘대로 불러서 사용할 수 있는 특별함수이고 우리는 이를 특별히 내장함수라고 호칭한다. 그렇다 `len` 함수는 내장함수이고, 용도는 그 이름에서 유추할 수 있는데, `length`(길이)의 약자로 만든 함수 이름으로 결국 특정 데이터가 보유하고 있는 데이터의 양(데이터의 길이)을 알려주는 함수이다. 따라서, 여기서 `len` 함수로 넘겨진 데이터값은 `numbers`라는 리스트로 결국, 해당 리스트가 갖고 있는 아이템의 개수를 알려주게 된다. 26번 라인의 코드에 의해서 사용자가 임의의 숫자 2개를 입력했으면, `len` 함수로 확인해 보면 현재 아이템개수는 2라는 것을, 사용자가 임의의 숫자 3개를 입력했으면 현재 아이템개수는 3이라는 것을 알 수 있게 된다.

위에서 각각 개별적으로 파악된 의미들을 종합해 해당 코드를 정확하게 다시 해석하면 ***len(numbers) != 0*** 이란 조건을 가진 ***while*** 문은 결국, ***numbers*** 라는 리스트에 ***아이템이 하나도 없을 때까지 반복***하라는 것이다. 사용자가 임의로 입력하는데로 numbers에는 계속 아이템들이 채워질 텐데 어쩌서 하나도 없는 상황이 가능한 것일까 라는 의문이 들 수 있다. 그것은 19~20번 라인의 코드를 해석하면 이유를 찾을 수 있다.

34 19~20번 라인의 코드는 사용자에게 의해 입력된 numbrers 리스트 안에 숫자값들을 하나하나씩 꺼내와 합계를 구하는 코드이다. 위에서 우리는 리스트를 다루는 방법 3가지(신규 값을 넣기, 기존 값을 읽기, 기존 값을 제거하기) 중에 1가지(신규 값을 넣기)를 배웠고, 이번엔 기존에 저장되어 있는 읽어들 수는 방법을 배울 차례이다. 이를 정확히 이해하려면 먼저, 문법과 더불어 리스트의 참조 인덱스(reference index)라는 개념에 대해 배워야 한다. 리스트의 특정 아이템을 읽는 문법은 다음과 같다.

리스트이름[참조 인덱스]

다음 예에서 볼 수 있듯이 리스트 안에 아이템이 존재하게 되는 그 순간(리스트 정의와 동시에 초기값 설정이든, 추후 신규 아이템이 추가되든), 곧바로, 다음에 그림과 같이 파이썬 언어 내부적으로 각 아이템이 리스트 내에 위치한 순서에 따라 맨 앞에서부터 0부터 1씩 증가하며 자동으로 매겨지는 우리 눈에는 보이지 않는 인덱스값이 내부적으로 존재하게 된다. 우리는 그 인덱스값을 이용해서 리스트 안의 각 아이템의 값들을 읽을 수 있게 된다.

```
words = ["apple", "banana", "grape"]
참조 인덱스:      0           1           2
```

우리가 만약 저 words라는 리스트 안에 두번째 위치한 "banana"라는 아이템(값)을 읽어오고 싶다면, 이제 리스트의 특정 위치의 아이템을 읽는 문법에 따라 words[1]이라고 표현해 읽을 수 있는 것이다. 아래 그림처럼 엔트리는 이러한 리스트 내의 값들과 참조 인덱스값들을 실시간으로 시각

화하여 보여주고 있는데, 우리의 예제에서 실행결과화면을 보면 좌측상단에 numbers 리스트 안에 2와 3이 순서대로 들어있는 것을 확인할 수 있다. 즉 사용자는 처음에 2라는 숫자값을 입력하고, 이어서 3이라는 숫자값을 입력한 상황인 것이다.



19라인의 코드는 numbers[0]이라는 문법표현으로 numbers 리스트 안에 맨 첫번째 아이템을 읽어서 그 값을 result라는 변수에 저장해 놓는 것이다. 바로 다음의 20번 라인 코드는 무엇인가? numbers 리스트 안에 위치한 numbers.pop()의 형식으로 pop 함수를 부르고 있는데, pop이라는 그 이름에서 알 수 있듯이 리스트 안에 특정위치의 데이터를 꺼내서 삭제하는 용도의 함수이다. 그래서, 이 함수의 사용은 리스트이름.pop(삭제하려는 아이템의 참조 인덱스) 형태로 호출해 사용할 수 있다. 따라서, 20번 라인의 numbers.pop(0)의 의미는 리스트 안에 맨 첫번째(인덱스 0에 의해) 아이템을 리스트로부터 삭제(제거)하라는 의미이다.

우리 예제의 위의 실행 결과화면에서(2, 3값이 연달아 입력되어진 상태에서) 연이어 살펴보면, 이 코드가 실행되면 리스트 안에 맨 처음값 숫자 2는 리스트에서 제거됨과 동시에 그 뒤에 위치해 있던 숫자 3이 방금 제거되어진 2가 있던 위치로 자동으로 끌어당겨져와 이제 맨앞에 위치하게 된다. pop(0)라는 의미는 결국 현 시점에서 항상 리스트 상에 맨 앞에 위치한 값을 가르키게 되는 것이다. 위에 코드를 엔트리에서 직접 실행시켜보자. 글로 읽은 것보다 한번 보는게 더 나운데 엔트리가 이러한 단계적 동작을 실시간으로 시각화해 비주얼적으로 보여주므로 직관적으로 이해할 수 있게 되기 때문이다.

예제 18-20번 라인의 코드를 통합적으로 다시 재해석하면, 현재 numbers 리스트에 저장되어 있는 아이템값을 하나씩 꺼내서 읽어 기존 합계(result변수값)에 더한 후에 그 읽은 값은 이제 리스트에서 제거해 버리는 일을 하되, 이런식으로 읽고 더하고 제거하는 일을 결국 리스트에 아무 아이템도 남지 않을 때까지 반복해서 수행하라는 의미이다. 즉, 사용자가 입력한 모든 값의 최종 합계를 구하고자하는 목적이며, 합계를 구하면서 동시에 리스트 안에 내용을 비워 깨끗하게 만드는 일을 함께 수행하고 있다.

다시 처음으로 돌아가 우리가 목표한 사용자로부터 제안없는 수를 연속적으로 입력받아 입력한 모든 수를 더한 합계를 구해주는 기능을 갖는 앱을 최종구현 완료하였다. 해당 기능은 반드시 위에서 언급된 식으로만 코딩해야 하는 것이 당연히 아니다. 동일 기능을 여러 가지 방법으로 구현할 수 있는데, 지금의 코드는 그 중에 한 방법일 뿐이다. 사실은 이게 엔트리-파이썬이 아니고, 오리지널 파이썬였다면 훨씬 더 간단하고 효과적으로 구현할 수 있는 코드가 존재하나, 우리는 엔트리-파이썬이 허용하는 문법표현 한계 안에서 코딩할 수 밖에 없고 또 여러분에게 리스트를 조작하는 다양한 표현법을 보여주려고 일부러 저렇게 코딩한 이유도 있다는 것을 참고하길 바란다.